

Transformers

Detailed Architecture Beyond Attention



The Paradigm Shift

Beyond Sequential Limits

Sequential vs. Parallel

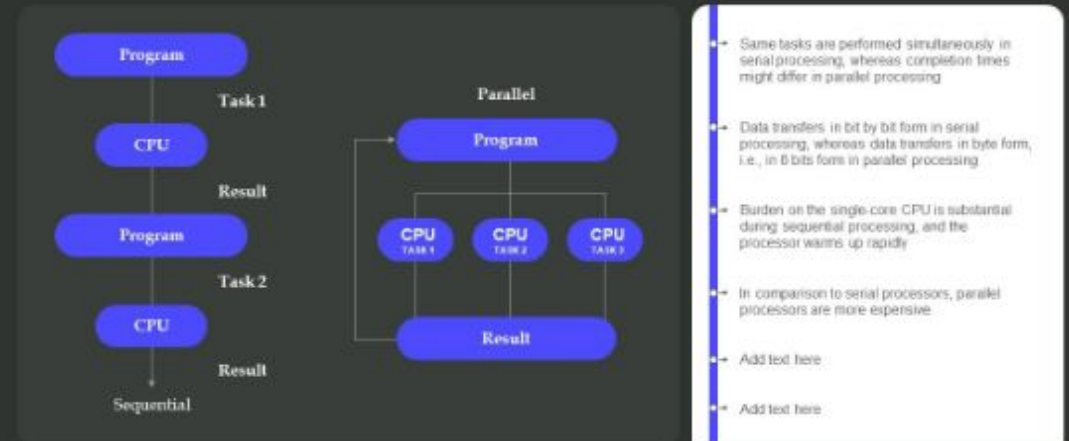
RNNs process text one word at a time, creating a bottleneck.

Transformers treat the entire sentence as a single data block.

- **No Recurrence:** No hidden state to pass forward.
- **Global Context:** Every word "sees" every other word simultaneously.
- **Hardware Optimized:** Designed specifically to leverage the power of modern GPUs.

Difference between serial and parallel processing

This slide shows the comparison between serial and parallel processing based on factors such as cost, processor heat, time to complete a task, data transfer rate, and so on. In serial processing, there is only one CPU to complete a task, whereas parallel processing comprises multiple CPUs.



This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

Unprecedented Efficiency

~10x

Training Speedup

Efficiency Through Parallelism

By removing the dependency on previous time steps, the model can compute gradients for all tokens at once. This allowed models to scale to trillions of tokens during pre-training, something impossible with traditional LSTM architectures.



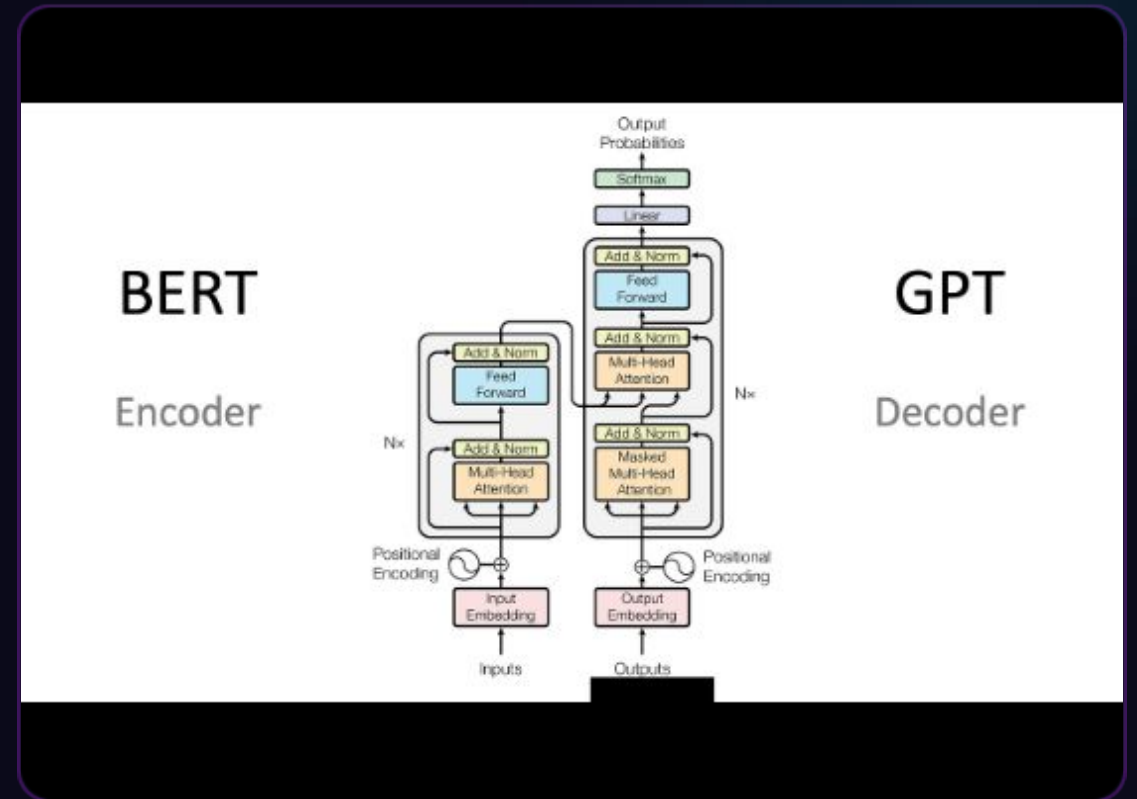
The Structural Backbone

The Encoder-Decoder Duo

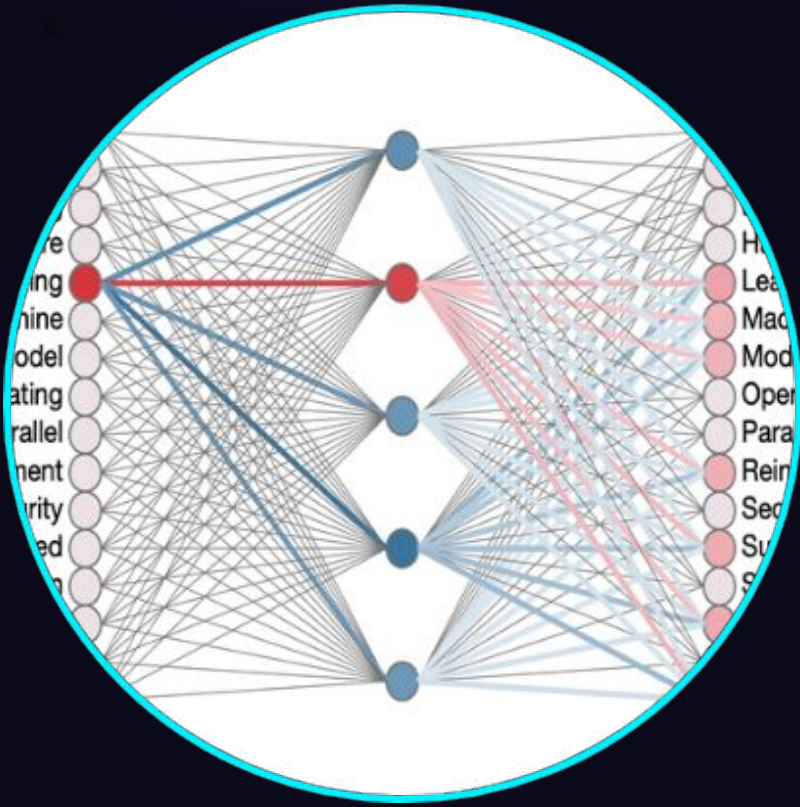
The Two Halves

Standard Transformers consist of two primary stacks: Encoders and Decoders.

- **Encoder:** Extracts features and context from the source input.
- **Decoder:** Generates output tokens one by one based on encoder context.
- **Symmetry:** Both consist of identical blocks stacked on top of each other (usually 6 to 96 layers).



Input: Tokens & Embeddings



Giving Words Meaning

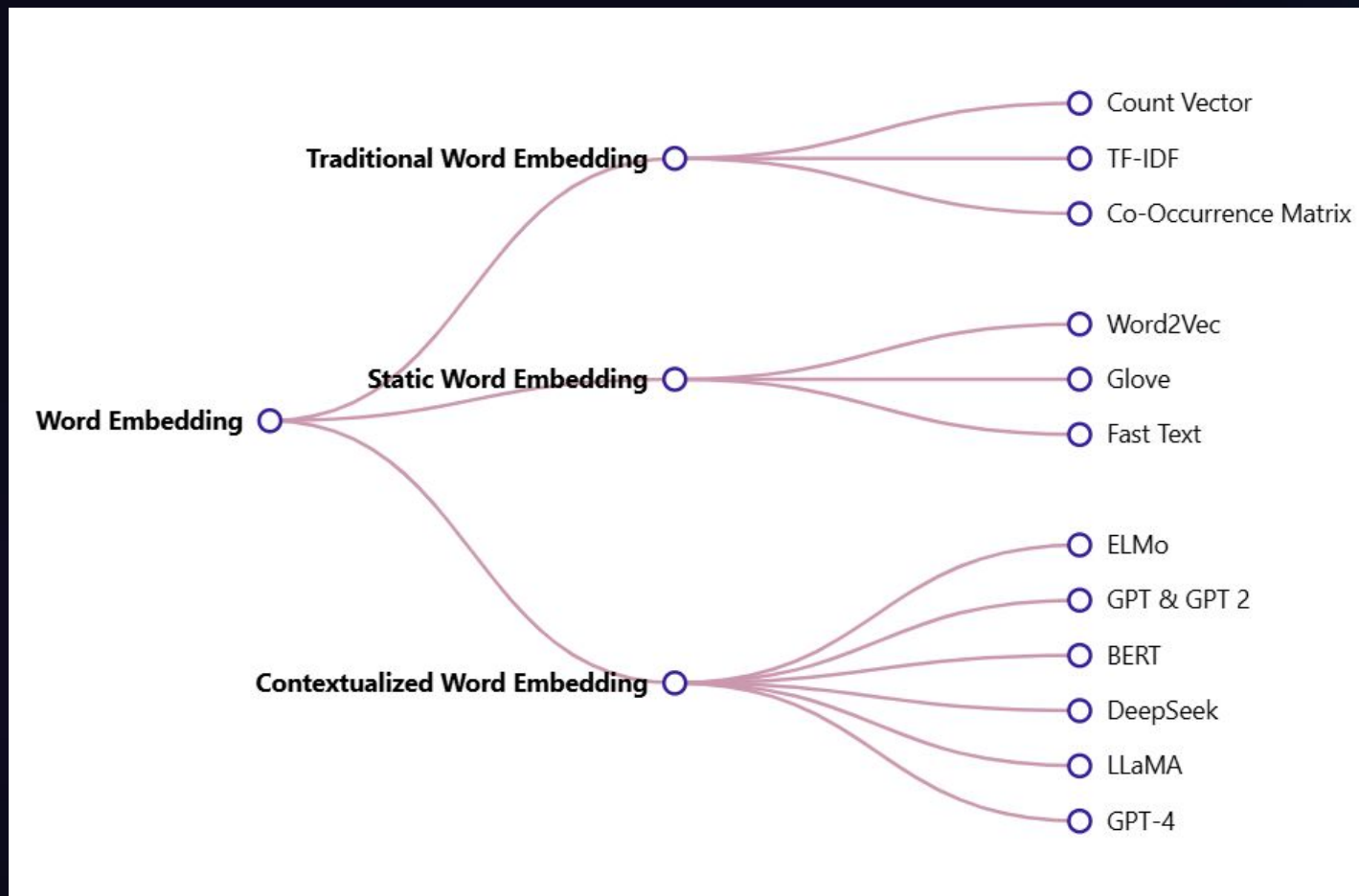
Computers don't read words; they read vectors. The embedding layer converts discrete tokens into high-dimensional mathematical space.

Vector Dimensions: Each word is represented by a vector (e.g., 512 dimensions). Words with similar meanings are positioned closer to each other in this space.

Input: Tokens & Embeddings

Giving Meaning

What makes embedding techniques different from each other, is how they approach this word → vector conversion.



Input: Tokens & Embeddings

Semantic Representation

Some types of embeddings capture the semantic relationship between words. This means that words with closer meanings or relationships are closer in the vector space than words that are less related. For example, the vectors of "cat" and "dog" must be more similar than "dog" and "strawberry".

Dimensionality

What should be the size of an embedding vector, 15, 50, 300? Striking the right balance is key. Smaller vectors (lower dimensions) are more efficient to keep in memory or to process, while bigger vectors (higher dimensions) can capture intricate relationships, but are prone to overfitting. For reference, GPT-2 model family has an embedding size of at least 768.

Example : Embeddings in Action (DeepSeek-R1-Distill-Qwen-1.5B)



token_id	token	Embedding Vector (1536 dimensions)
151646		-0.027466, 0.002899, -0.005188 ... 0.021606
5835	HTML	-0.018555, 0.000912, 0.010986 ... -0.015991
20329	#cod	-0.026978, -0.012939, 0.021362 ... 0.042725
388	ers	-0.012085, 0.001244, -0.069336 ... -0.001213
525	#are	-0.001785, -0.008789, 0.006195 ... -0.016235
537	#not	0.016357, -0.039062, 0.045898 ... 0.001686
6509	#considered	-0.000721, -0.021118, 0.027710 ... -0.051270
54846	#programmers	-0.047852, 0.057861, -0.069336 ... 0.005280

Solving the Order Problem

Positional Encoding

Since Transformers don't process words sequentially, they "forget" word order. We fix this by adding a "positional vector" to the embedding.

- 📏 **Sinusoidal Functions:** Using sine and cosine waves of different frequencies to encode position.
- ⊕ **Additive Nature:** These vectors are added directly to the word embeddings, preserving both meaning and position.
- ∞ **Infinite Sequences:** This mathematical trick allows the model to generalize to sequence lengths it hasn't seen during training.

$$\text{PE}(\text{pos}, 2i) = \sin \frac{\text{pos}}{10000^{2i / d_{\text{model}}}}$$

Core Sub-Layers

Component	Primary Function	Key Benefit
Residual Connections	Adds input to sub-layer output	Prevents vanishing gradients in deep stacks
Layer Norm	Normalizes neuron activations	Stabilizes training and speeds up convergence
Feed Forward	Point-wise linear transformations	Allows the model to "process" the information found
Softmax Output	Converts logits to probabilities	Determines the final chosen word

The Transformation Block



Add & Norm

Every sub-layer is wrapped in a "Residual Connection" followed by "Layer Normalization". This keeps the signal clear as it passes through 100+ layers.



Feed Forward

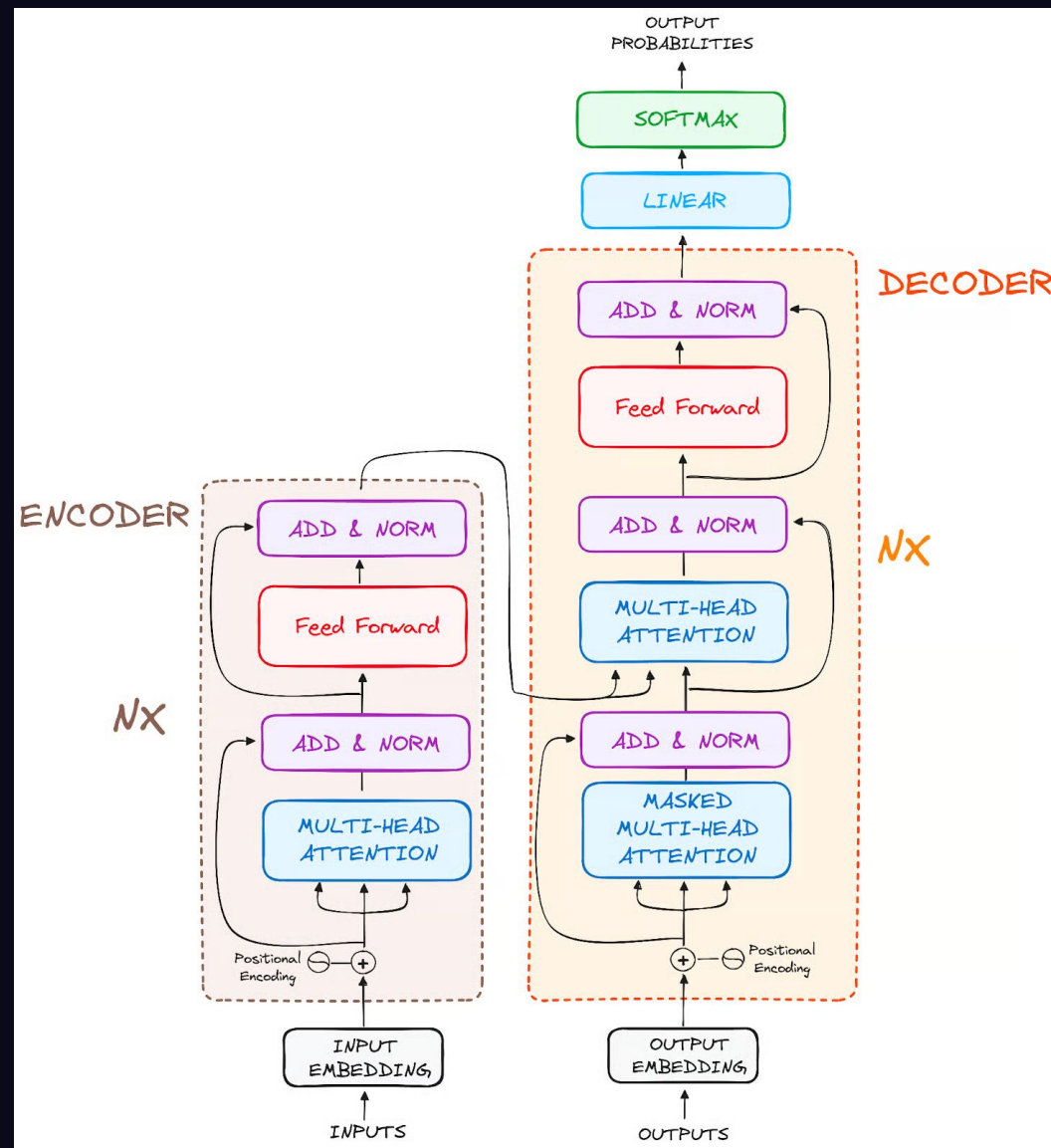
Consists of two linear layers with a non-linear activation (like ReLU or GeLU). This is applied to each position separately and identically.



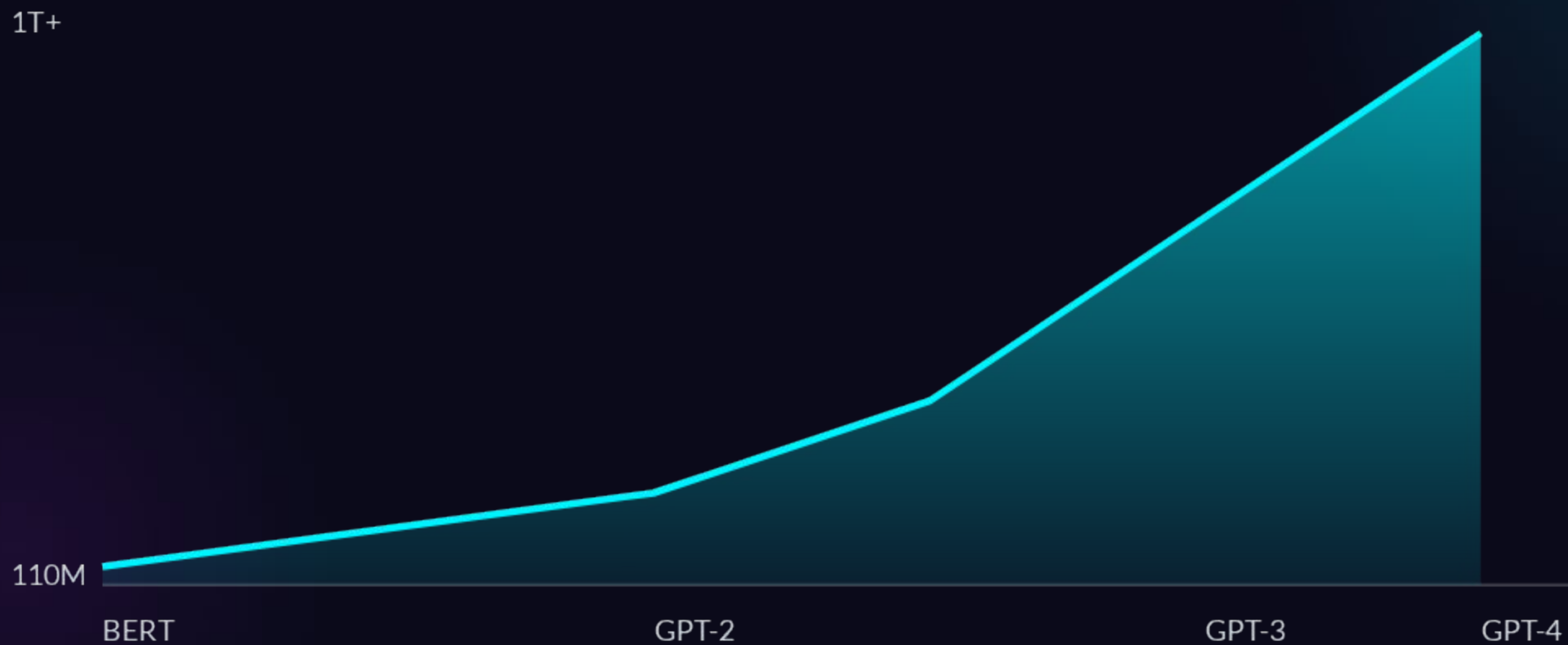
Output Projections

The final layer projects the internal hidden state back into the vocabulary space to predict the next token in the sequence.

The Transformation Block

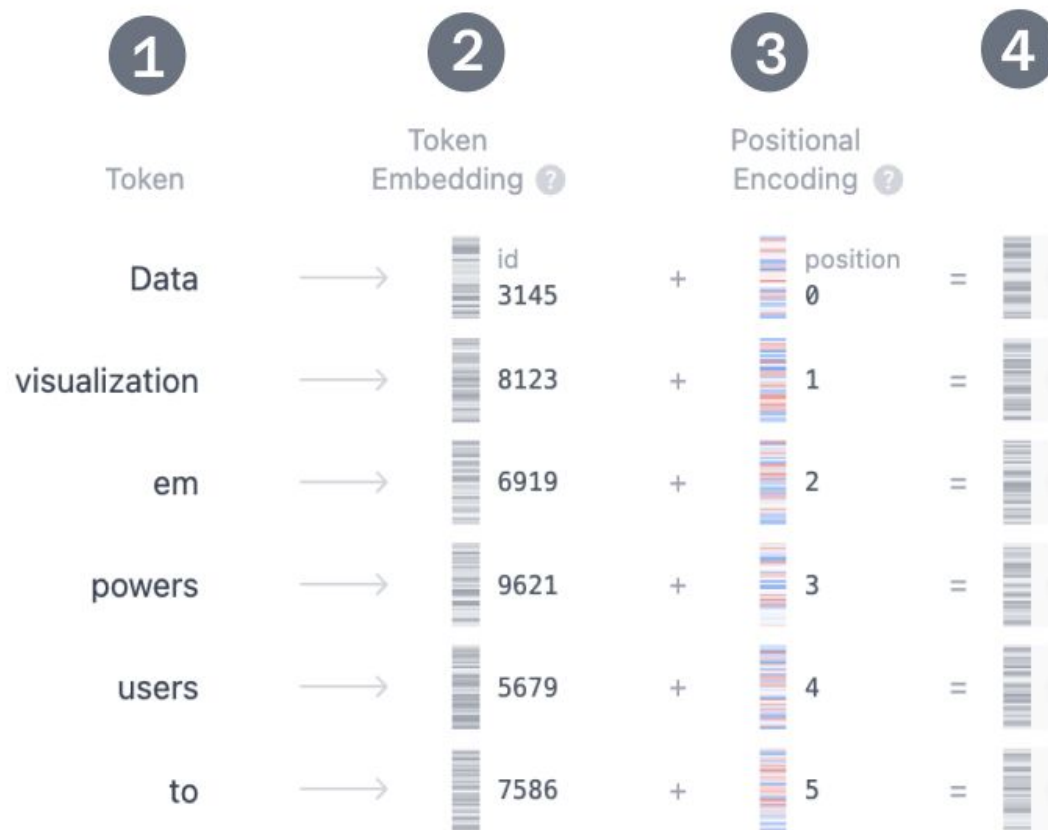


Scaling the Modern Giants

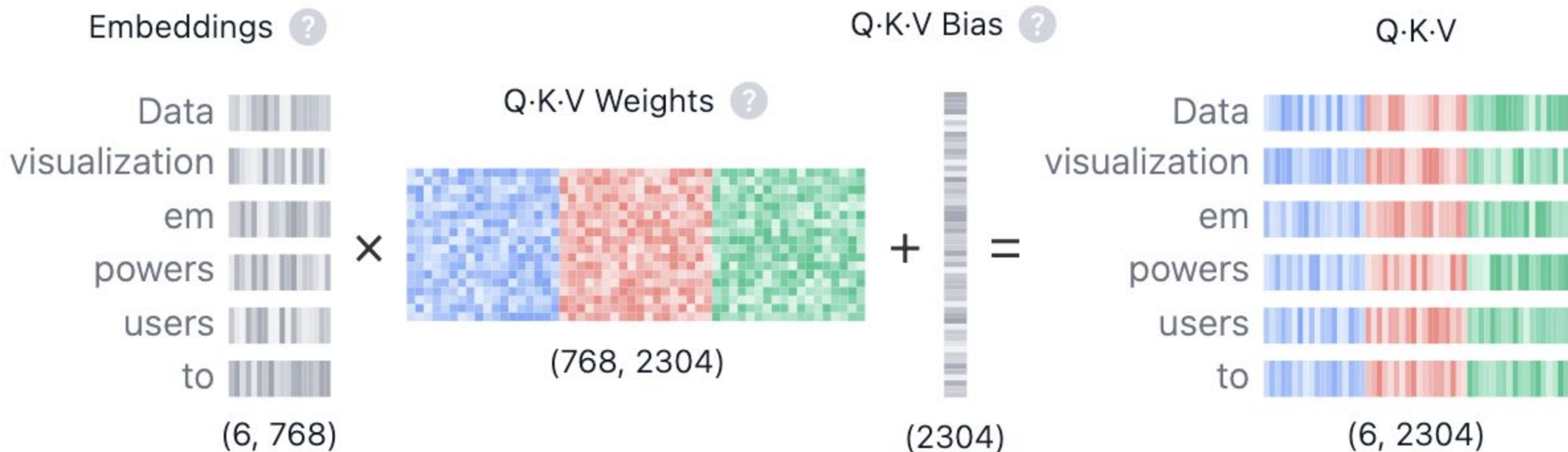


The architecture's scalability allows it to handle exponentially larger datasets and parameter counts.

Example



Attention Layers



$$QKV_{ij} = \left(\sum_{d=1}^{768} \text{Embedding}_{i,d} \cdot \text{Weights}_{d,j} \right) + \text{Bias}_j$$

The "Translator" of Neural Networks

The Softmax function is the **critical final layer** of LLM architecture. It maps raw numerical scores (logits) to a valid probability distribution.

$$\text{Softmax} (x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

- ▶ **K** represents the number of words in the vocabulary.
- ▶ Ensures the model can make **probabilistic predictions** for the next token.

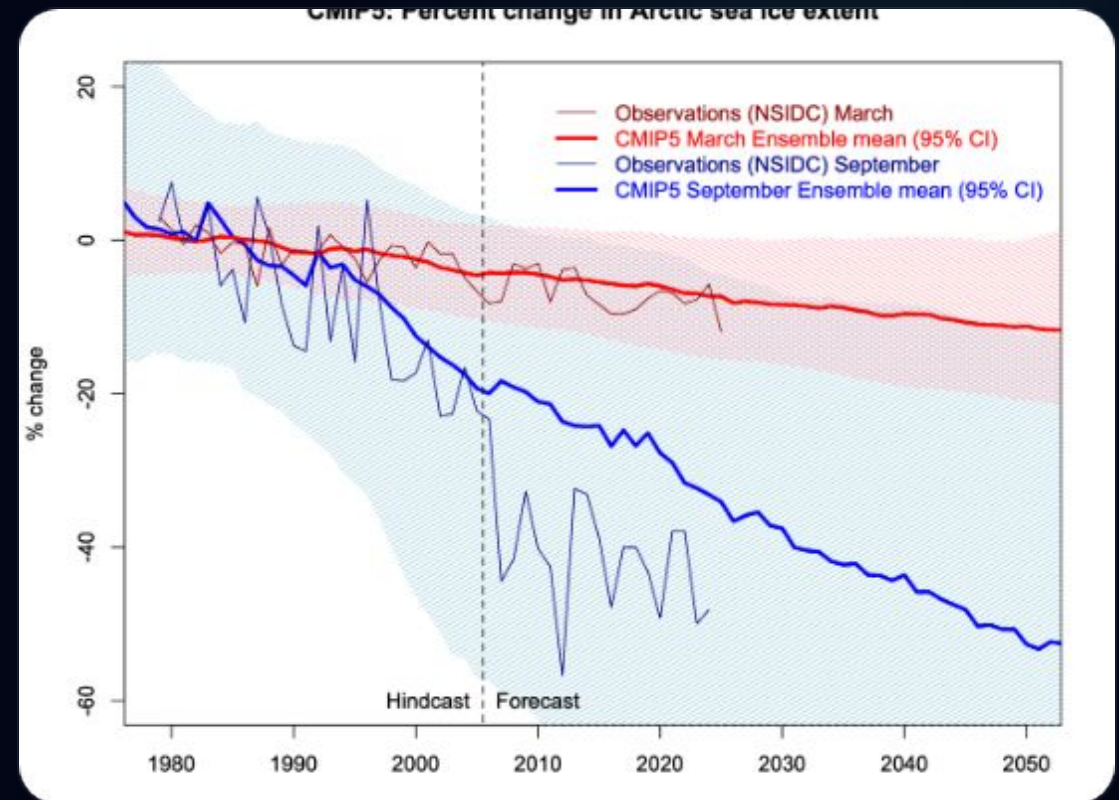


Temperature (T): Bounciness Control

Temperature is a hyperparameter that scales logits *before* the Softmax function is applied, directly influencing the final randomness.

$$P_i = \frac{e^{z_i / T}}{\sum_j e^{z_j / T}}$$

- > **Low T (< 1):** Sharpens the distribution. The model becomes more deterministic, favoring the most likely tokens.
- > **High T (> 1):** Flattens the distribution. Less likely tokens gain "bounciness," increasing creativity but risking hallucinations.



Output

Temperature ? Sampling ? ☒ Top-k ☐ Top-p				
0.8 k5				
Tokens	Logits	Scaled logits	Top-k	Softmax
visualize	-135.91	-169.89	-169.89	54.67%
create	-136.68	-170.85	-170.85	20.87%
see	-137.12	-171.40	-171.40	12.09%
make	-137.65	-172.06	-172.06	6.26%
easily	-137.67	-172.08	-172.08	6.11%
quickly	-137.72	-172.15	-∞	0%
explore	-137.78	-172.23	-∞	0%
find	-138.05	-172.56	-∞	0%

Thank You!